Worksheet 6

Student Name:Suyash **UID:25MCI10054**
Branch:MCA (AI&ML) **Section/Group:MAM-1 A**
Semester:2nd **Date of Performance:03/03/2026**
Subject Name:- Technical Training -I **Subject Code: 25CAP-652**

Implementation of Views in PostgreSQL

Aim

To learn how to create, query, and manage views in PostgreSQL to simplify database queries and provide a layer of abstraction for end-users.

Tools Used

● PostgreSQL

Objectives

- To understand how views provide data abstraction
- To learn how to restrict access to sensitive data using views
- To simplify complex queries using virtual tables
- To understand creation, replacement, and deletion of views
- To relate view concepts to real-world applications

Theory

A View is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".

- **Simple Views**: Created from a single table without any aggregate functions or grouping. These are often updatable.
- **Complex Views**: Created from multiple tables using JOINs, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.
- **Security Layer**: In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.
- **Benefits**: They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

Experiment Steps

Table Creation

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);

CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(50),
salary INT,
status VARCHAR(20),
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

INSERT INTO departments VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance');

INSERT INTO employees VALUES
```

(101, 'Sumit', 45000, 'Active', 2),
(102, 'Neha', 52000, 'Active', 1),
(103, 'Rahul', 38000, 'Inactive', 2),
(104, 'Priya', 60000, 'Active', 3),
(105, 'Karan', 48000, 'Active', 1);

Table Employees

	emp_id [PK] integer	emp_name character varying (50)	salary integer	status character varying (20)	dept_id integer
1	101	Sumit	45000	Active	2
2	102	Neha	52000	Active	1
3	103	Rahul	38000	Inactive	2
4	104	Priya	60000	Active	3
5	105	Karan	48000	Active	1

Table Departments

	dept_id [PK] integer	dept_name character varying (50)
1	1	HR
2	2	IT
3	3	Finance

Step 1 : Creating a Simple View for Data Filtering

```
CREATE VIEW active_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE status = 'Active';

SELECT * FROM active_employees;
```

Output

	emp_id integer	emp_name character varying (50)	salary integer
1	101	Sumit	45000
2	102	Neha	52000
3	104	Priya	60000
4	105	Karan	48000

Step 2: Creating a View Using JOIN (Multiple Tables)

```
CREATE VIEW employee_department AS
SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id;
```

```
SELECT * FROM employee_department;
```

Output

	emp_id integer	emp_name character varying (50)	dept_name character varying (50)	salary integer
1	101	Sumit	IT	45000
2	102	Neha	HR	52000
3	103	Rahul	IT	38000
4	104	Priya	Finance	60000
5	105	Karan	HR	48000

Step 3: Creating an Advanced Summarization View

```
CREATE VIEW department_salary_summary AS
SELECT d.dept_name,
COUNT(e.emp_id) AS total_employees,
AVG(e.salary) AS avg_salary,
MAX(e.salary) AS highest_salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

```
SELECT * FROM department_salary_summary;
```

Output

	dept_name character varying (50)	total_employees bigint	avg_salary numeric	highest_salary integer
1	Finance	1	60000.000000000000	60000
2	IT	2	41500.000000000000	45000
3	HR	2	50000.000000000000	52000

Learning Outcome

- Understand how views provide abstraction and simplify queries
- Learn to create simple and complex views
- Implement views for data security and restricted access
- Use views for real-world reporting and analysis
- Demonstrate proper syntax for view management

Result

This experiment demonstrates how views can be used to simplify complex SQL queries, enhance security by restricting sensitive data, and provide a logical layer for end-users. Students gain practical knowledge of creating, querying, updating, and dropping views in PostgreSQL for enterprise-level applications.